

1. Objectives:

1. To implement Breadth First Search (BFS) using queue in Graph.
2. To implement Depth First Search (DFS) using stack in Graph.

2. Introduction:

- **Graph:** Graph is a non-linear data structure consisting of a set of vertices (nodes) and edges that connect pairs of vertices. In this lab, the graph is represented using an adjacency matrix, where a 2D array is used to store connections between vertices. If there is an edge between two vertices, the corresponding cell in the matrix is marked as 1, otherwise, it is 0.

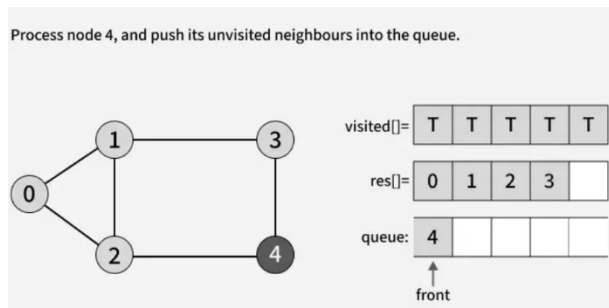


Fig 10.1: BFS Search

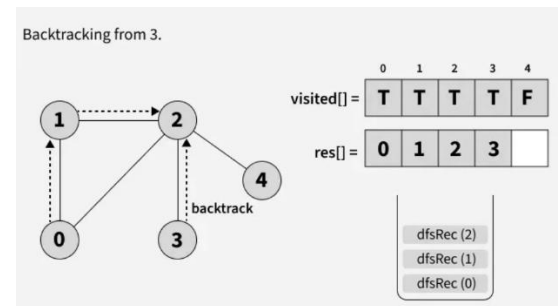


Fig 10.2: DFS Search

- **BFS Search:** Breadth First Search (BFS) is a graph traversal technique where nodes are explored level by level. It starts from a given node and first visits all its immediate neighbors before moving to the next level of nodes. BFS uses a queue data structure to maintain the order of traversal. This ensures that nodes are visited in the order they are discovered, following the First In First Out (FIFO) principle.
- **DFS Search:** Depth First Search (DFS) is a another traversal technique where nodes are explored as deeply as possible along each branch before backtracking. It starts from a node and continues moving to an unvisited adjacent node until no further nodes are available, then it backtracks to explore other paths. DFS is commonly implemented using recursion, which internally uses a stack to manage function calls.

3. Procedure

Experiment 1: Graph BFS and DFS Search

Input:

- $V \rightarrow$ Number of vertices
- $\text{adj}[V][V] \rightarrow$ Adjacency matrix
- $\text{start} \rightarrow$ Starting node

Output:

- Adjacency matrix
- BFS traversal
- DFS traversal

Algorithm:

addEdge(adj, u, v):

1. $\text{adj}[u][v] \leftarrow 1$
2. $\text{adj}[v][u] \leftarrow 1$

BFS(adj, V, start):

1. $\text{visited}[0 \dots V-1] \leftarrow \text{false}$

2. create empty queue q
3. $\text{visited}[\text{start}] \leftarrow \text{true}$
4. push start into q
5. **while** q is not empty **do**
6. node $\leftarrow$ front element of q
7. pop q
8. print node
9. **for** i $\leftarrow$ 0 to V-1 **do**
10. **if** $\text{adj}[\text{node}][i] == 1$ AND $\text{visited}[i] == \text{false}$ **then**
11. $\text{visited}[i] \leftarrow \text{true}$
12. push i into q

DFS(adj, V, start):

- $\text{visited}[0 \dots V-1] \leftarrow \text{false}$
- call DFS2(adj, V, start, visited)

DFS2(adj, V, current, visited):

- $\text{visited}[\text{current}] \leftarrow \text{true}$
- print current
- **for** i $\leftarrow$ 0 to V-1 **do**
- **if** $\text{adj}[\text{current}][i] == 1$ AND $\text{visited}[i] == \text{false}$ **then**
- call DFS2(adj, V, i, visited)

4. Required Hardware and Software:

- **Hardware:** AMD RYZEN 7, RAM 8 GB
- **Software:** Visual Studio Code 1.109

5. Experiment Implementation:

Experiment 1: Graph BFS and DFS Search

```
#include <iostream>
#include <queue>
#include <stack>
using namespace std;
class Graph{
public:
    int V;
    int adj[5][5];
    Graph(int size){
        V = size;
        for (int i = 0; i < V; i++){
            for (int j = 0; j < V; j++){
                adj[i][j] = 0;
            }
        }
    }
    void addEdge(int u, int v){
        adj[u][v] = 1;
        adj[v][u] = 1;
    }
    void display(){
        for (int i = 0; i < V; i++){
            for (int j = 0; j < V; j++){
                cout << adj[i][j] << " ";
            }
            cout << endl;
        }
    }
    void bfs(int start){
        bool visited[5] = {false};
        queue<int> q;
        visited[start] = true;
        q.push(start);

        while (!q.empty()){
```

```

        int node = q.front();
        q.pop();
        cout << node << " ";

        for (int i = 0; i < V; i++){
            if (adj[node][i] == 1 && visited[i] == false){
                visited[i] = true;
                q.push(i);
            }
        }
    }
}

void dfs2(int current, bool visited[]){
    visited[current] = true;
    cout << current << " ";
    for (int i = 0; i < V; i++){
        if (adj[current][i] == 1 && !visited[i]){
            dfs2(i, visited);
        }
    }
}

void dfs(int start){
    bool visited[5] = {false};
    dfs2(start, visited);
}

};

int main(){
    Graph g(5);
    g.addEdge(0, 1);
    g.addEdge(1, 2);
    g.addEdge(1, 3);
    g.addEdge(2, 4);
    g.display();
    g.bfs(0);
    cout << endl;
    g.dfs(0);
    return 0;
}

```

Table 10.1: Experiment 1 output

Input	Expected Output	Actual Output
	0 1 0 0 0	0 1 0 0 0
	1 0 1 1 0	1 0 1 1 0
	0 1 0 0 1	0 1 0 0 1
	0 1 0 0 0	0 1 0 0 0
	0 0 1 0 0	0 0 1 0 0
	0 1 2 3 4	0 1 2 3 4
	0 1 2 4 3	0 1 2 4 3

6. Discussion & Conclusion:

In this lab, we learned graph representation using an adjacency matrix and performed traversal using both BFS and DFS techniques. BFS helped to understand level-wise traversal using a queue, while DFS demonstrated depth-wise traversal using recursion. The use of a visited array was essential to prevent revisiting nodes and avoid infinite loops. Some challenges were faced in understanding the flow of traversal and maintaining correct conditions for visiting nodes, especially in DFS recursion. Logical tracing was required to clearly follow how nodes are visited in different orders.

7. Reference:

1. <https://www.geeksforgeeks.org/dsa/breadth-first-search-or-bfs-for-a-graph/> accessed at 6.45 PM on 5th May 2026.
2. <https://www.geeksforgeeks.org/dsa/depth-first-search-or-dfs-for-a-graph/> accessed at 07:25 Pm on 5th May 2026.